

AI Failure Intelligence pada Pipeline CI/CD TaskFlow

FINAL PROJECT
OPERASIONAL
PENGEMBANG

PRESENTED BY
KELOMPOK 3

DAFTAR ISI

1

**LATAR
BELAKANG**

2

**LANDASAN
RISET**

3

ARSITEKTUR

4

EVALUASI

ANGGOTA

NATHAN KHO PANCRAS

--->

5027231002

DAFFA RAJENDRA P

--->

5027231009

MAULANA AHMAD ZAHIRI

--->

5027231010

KEVIN ANUGERAH FAZA

--->

5027231027

RM. NOVIAN MALCOLM B

--->

5027231035

AZZA FARICHI T

--->

5027231071

NABIEL NIZAR A

--->

5027231087

Latar Belakang: Mengapa Raw Log Saja Tidak Cukup?

01

Masalah Log Mentah (Raw Logs):

- Sangat panjang dan dipenuhi noise (setup runner, download dependencies).
- Mengharuskan developer mencari baris error secara manual.

Masalah Retensi & Memori:

- Log di GitHub Actions bersifat sementara (terhapus otomatis setelah batas retensi).
- Insight dari kegagalan serupa di masa lalu sering kali terlupakan.

Masalah Kecepatan Diagnosis:

- Developer membuang waktu mencari solusi di Google/StackOverflow saat pipeline gagal.



Near-Duplicate Build Failure Detection from Continuous Integration Logs

Improving the Prediction of Continuous Integration Build Failures Using Deep Learning

Saidani, Ouni, dan Mkaouer, 2022, Automated Software Engineering, Springer.

Masalah

- Failed dan passing build logs memiliki banyak baris yang sama.
- Rata-rata satu build log berisi sekitar 9.000 baris.
- Raw-log similarity mudah dipengaruhi setup dan dependency noise.

Metodologi

- Dataset: 410 GitHub Actions build logs.
- 71 failed builds dan 339 passing builds.
- Near-duplicate dilabeli berdasarkan failed tests yang sama.
- Menggunakan Jaccard, cosine, containment, dan compression distance.
- OOVD digunakan untuk menonjolkan failure-relevant lines.
- Evaluasi menggunakan Top-K retrieval dengan $K = 5$.

Hasil

Metrik	Tanpa OOVD	Dengan OOVD
Precision@K	0.526	0.864
MAP@K	0.814	0.941

Landasan Riset

Adaptasi pada TaskFlow

- Log cleaning sebelum retrieval.
- OOVD-inspired suspicious-line extraction.
- Top-3 retrieval menggunakan Gemini embedding dan cosine similarity.
- OOVD digunakan sebagai supporting evidence, bukan classifier.

Improving the Prediction of Continuous Integration Build Failures Using Deep Learning

Improving the Prediction of Continuous Integration Build Failures Using Deep Learning

Saidani, Ouni, dan Mkaouer, 2022, Automated Software Engineering, Springer.

Landasan Riset

Masalah

- Historical build outcomes memiliki pola temporal.
- Failed builds lebih sedikit daripada passing builds sehingga terjadi class imbalance.

Metodologi

- Dataset: 91.330 CI build records.
- Berasal dari 10 proyek open source berbasis Travis CI.
- Model utama: LSTM-RNN.
- Genetic Algorithm digunakan untuk hyperparameter optimization.
- Threshold moving digunakan untuk menangani class imbalance.
- Dibandingkan dengan Random Forest, Decision Tree, AdaBoost, Logistic Regression, dan SVC.

Hasil

Metrik	Median
AUC	72%
F1-score	57%
Accuracy	78%

Adaptasi pada TaskFlow

- Tidak melakukan training LSTM karena hanya memiliki sedikit historical records.
- Tidak memprediksi build berikutnya.
- Menggunakan historical failure sebagai knowledge untuk diagnosis setelah pipeline gagal.
- Synthetic validated logs digunakan untuk mengatasi cold-start.
- Real failure dikumpulkan bertahap melalui branch failure-history.

Catatan

Paper pertama mengukur Precision@K dan MAP@K pada near-duplicate retrieval. Paper kedua mengukur prediction dengan AUC, F1, dan accuracy. TaskFlow mengukur Top-1, Top-3, dan MRR (Mean Reciprocal Rank) karena implementasinya adalah retrieval, bukan build prediction classifier.

Arsitektur Sistem: AI Failure Intelligence

05

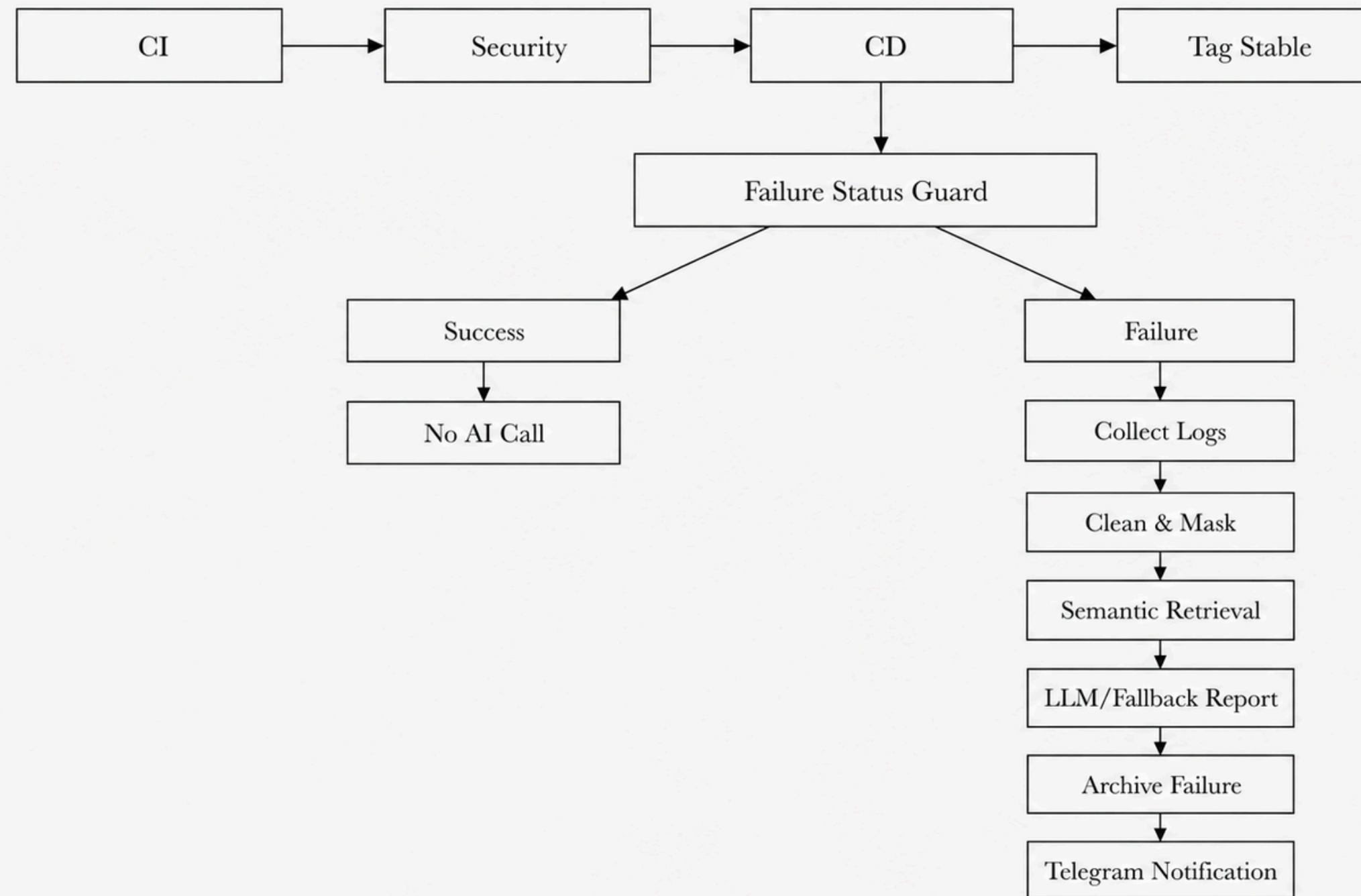


Fig 1. Arsitektur Pipeline

Arsitektur Sistem: AI Failure Intelligence

05

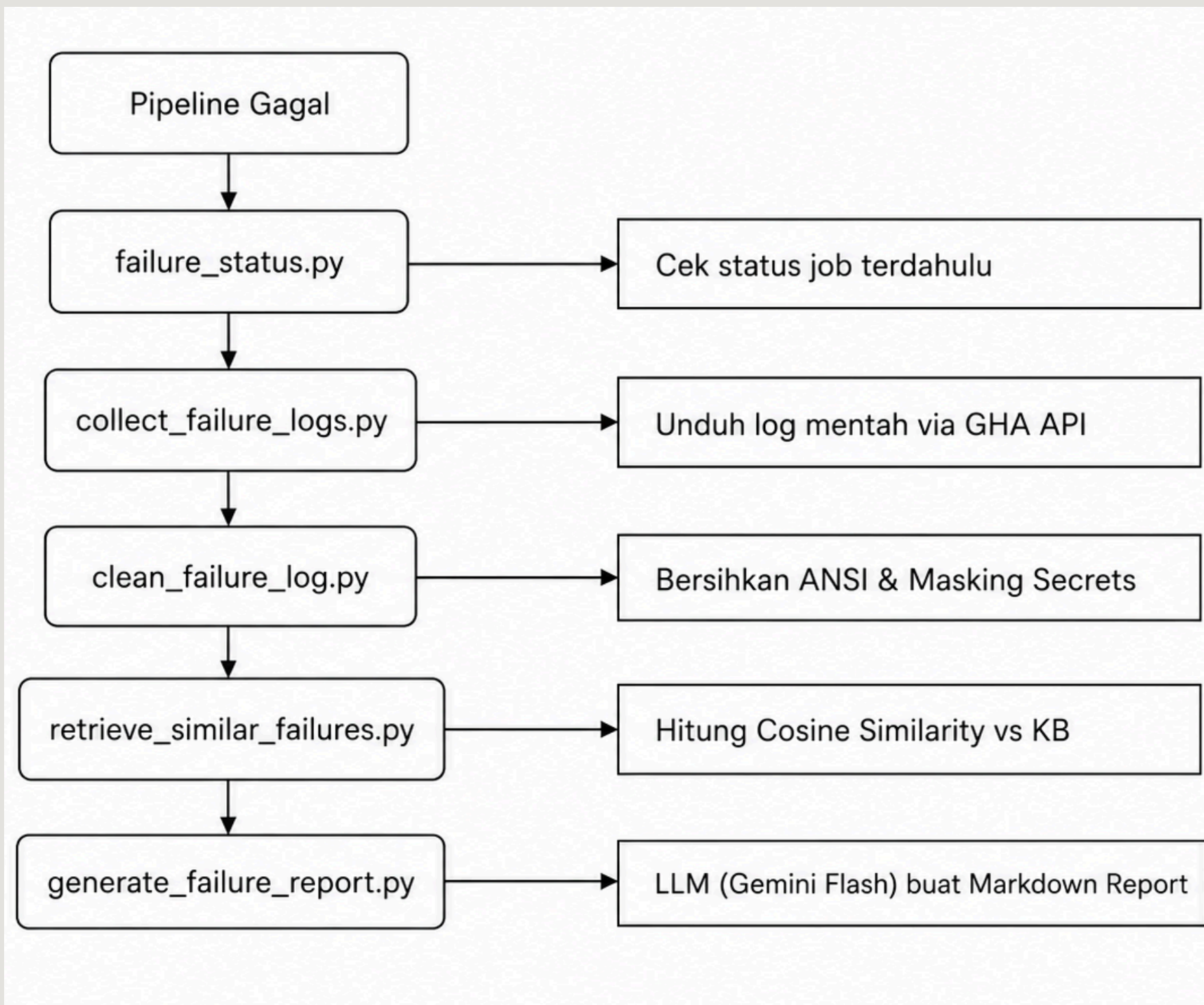


Fig 2. Arsitektur Failure intelligence

Komponen 1: Log Preprocessing & OOVD-Inspired Scoring

Log Cleaning & Sanitasi:

- Menghapus ANSI escape code, timestamp tertentu, dan baris kosong.
- Menyensor API key, token, password, dan Bearer credential.
- Membatasi log panjang dan memprioritaskan baris failure-relevant.

OOVD-Inspired Scoring:

- Membandingkan token log dengan vocabulary CI umum.
- Menandai baris yang mengandung token tidak umum.
- Menyimpan evidence ke oovd-lines.json.
- Bukan trained OOVD model dan bukan classifier utama.

Komponen 2: Failure Knowledge Base & Retrieval

05

Struktur Knowledge Base:

- Disimpan permanen di folder 'failure-knowledge-base/'.
- Berisi logs, penjelasan (notes), dan embedding index.

5 Kategori Kegagalan Awal (Cold-Start):

- `001-coverage-gate`: Nilai testing coverage di bawah batas 75%.
- `002-docker-build`: Kesalahan path/perintah Docker build.
- `003-integration-postgres`: Koneksi database PostgreSQL bermasalah.
- `004-security-scan`: vulnerability terdeteksi pada dependency.
- `005-smoke-test-health`: Endpoint deploy gagal merespons.

Pencarian Semantik:

- Embedding query dibuat dengan model `gemini-embedding-2` (768 dimensi).
- Diperbandingkan secara lokal dengan Cosine Similarity untuk mencari Top-3 error termirip.

Komponen 3: LLM Explanation Layer & Fallback

05

LLM Sebagai Penyusun Penjelasan (Bukan Classifier):

- Model: *gemini-3.1-flash-lite*.
- Menggunakan input dari **cleaned log**, **top-3 similar errors**, dan **OOVD suspicious lines**.
- Menghasilkan laporan: **Likely Root Cause**, **Evidence log**, dan **Suggested Debugging Steps** yang riil.
- Jika bukti log tidak cukup, LLM diinstruksikan menulis "evidence belum cukup".

API Failure Safety :

- Exponential backoff retry (hingga 5x) untuk kendala API.
- Otomatis beralih ke Fallback Report lokal jika API Key kosong/mati



Komponen 4: Automatic Failure Archive

- Archive berjalan setelah retrieval dan report.
- Hanya sanitized cleaned log yang disimpan.
- ID berasal dari GitHub run ID dan run attempt.
- SHA-256 digunakan untuk mendeteksi duplicate content.
- Entry baru: `source_type: real`, `validated: false`.
- AI report tidak dianggap sebagai ground truth.
- Hanya entry tervalidasi yang masuk embedding index.
- Concurrency mencegah dua workflow menulis history bersamaan.

- Tujuan**
- Mengukur tingkat akurasi retrieval kemiripan log sebelum diaplikasikan ke sistem produksi.

- 15 Repeated Controlled Scenarios:**
- 5 kategori utama, masing-masing dibuat 3 variasi log berbeda

- 2 Mode Pengujian:**
- ``full_cleaned_log``: Query menggunakan seluruh log yang sudah dibersihkan.
 - ``oovd_focused_log``: Query hanya menggunakan baris log ber-score OOVd tinggi.

- Metrik Evaluasi:**
- Top-1 Accuracy, Top-3 Accuracy, MRR (Mean Reciprocal Rank), dan Mean Similarity Score.

Table 1: Perbandingan Kinerja Retrieval pada Dua Mode Pemrosesan Log		
Metrik Evaluasi	Mode 1: Full Cleaned Log	Mode 2: OOVd-Focused Log
Top-1 Accuracy	15/15 (100%)	14/15 (93.3%)
Top-3 Accuracy	15/15 (100%)	15/15 (100%)
MRR (Mean Reciprocal Rank)	1.0000	0.9667
Mean Top-1 Similarity	0.8559	0.8192

Metodologi Evaluasi

Scenario	Expected	Full Top-1	Full Expected Rank	OOVD Top-1	OOVD Expected Rank	OOVD Lines
coverage-gate	coverage-gate	coverage-gate	1	coverage-gate	1	2
coverage-gate-unicode	coverage-gate	coverage-gate	1	coverage-gate	1	2
coverage-gate-new-package	coverage-gate	coverage-gate	1	coverage-gate	1	3
docker-build	docker-build	docker-build	1	docker-build	1	6
docker-build-go-mod	docker-build	docker-build	1	docker-build	1	5
docker-build-copy-path	docker-build	docker-build	1	docker-build	1	6
postgres-timeout	integration-postgres	integration-postgres	1	integration-postgres	1	4
postgres-auth	integration-postgres	integration-postgres	1	integration-postgres	1	4
postgres-migration	integration-postgres	integration-postgres	1	integration-postgres	1	5
security-scan	security-scan	security-scan	1	security-scan	1	5
security-gosec	security-scan	security-scan	1	security-scan	1	5
security-govulncheck	security-scan	security-scan	1	security-scan	1	3
smoke-test-health	smoke-test-health	smoke-test-health	1	smoke-test-health	1	5
smoke-test-stats	smoke-test-health	smoke-test-health	1	smoke-test-health	1	4
smoke-test-container-crash	smoke-test-health	smoke-test-health	1	integration-postgres	2	5

Fig 3. Hasil Testing Evaluasi

AI Failure Intelligence Report

Status

Failed

Failed Stage

CI

Current Failure Summary

Pipeline mengalami kegagalan pada semua job `CI` (Go 1.21, 1.22, dan 1.23) karena sistem gagal mengunduh log dengan error `HTTPError`.

Most Similar Failures

- 1. `synthetic-001-coverage-gate` (Similarity: 0.7262)
- 2. `synthetic-003-postgres-timeout` (Similarity: 0.7034)
- 3. `synthetic-004-security-scan` (Similarity: 0.6993)

Likely Root Cause

evidence belum cukup

Evidence

Log hanya menunjukkan `HTTPError` saat proses pengunduhan log job. Tidak ada informasi spesifik mengenai penyebab kegagalan internal pada job tersebut karena log itu sendiri tidak dapat diakses oleh sistem.

Suggested Debugging Steps

- 1. Periksa status API atau service penyedia log untuk memastikan tidak ada gangguan pada endpoint pengunduhan.
- 2. Lakukan verifikasi konektivitas jaringan dari runner ke server log.
- 3. Coba jalankan ulang job secara manual untuk melihat apakah `HTTPError` bersifat sementara (flaky).
- 4. Periksa apakah ada limitasi pada storage atau kuota akses log yang menyebabkan request ditolak.

Notes and Limits

Analisis ini terbatas pada kegagalan sistem pengunduhan log. Karena log utama dari proses build tidak tersedia, root cause spesifik dari kegagalan kode atau test tidak dapat ditentukan.

Fig 4-5. Hasil Report AI

Demonstrasi

Refleksi Kelompok: AI Failure Intelligence for CI/CD Pipeline

1. Hal yang paling mengejutkan dan pengaruhnya terhadap implementasi

log build yang gagal dan berhasil memiliki banyak bagian yang sama sehingga perbandingan raw log secara langsung kurang efektif. Li et al. (2025) dan juga menunjukkan bahwa proses filtering noise dan preprocessing log memberikan peningkatan akurasi yang signifikan.

Temuan ini mendorong implementasi untuk lebih berfokus pada pembersihan log, penghapusan informasi yang tidak relevan, dan ekstraksi bagian error sebelum proses retrieval dilakukan. Selain itu, penggunaan pretrained embedding dipilih karena lebih realistis dibanding melatih model dari awal dengan dataset yang terbatas.

Refleksi Kelompok: AI Failure Intelligence for CI/CD Pipeline

2. Perbedaan implementasi dengan paper

Implementasi yang dibuat tidak mengikuti pendekatan deep learning yang digunakan oleh Saidani et al. (2022) karena tujuan sistem adalah membantu diagnosis setelah build gagal, bukan memprediksi kegagalan build. Proses retrieval juga disederhanakan dengan menggunakan cosine similarity pada embedding dibanding menggabungkan beberapa metrik similarity seperti pada Li et al. (2025).

Untuk filtering log, digunakan pendekatan OOVD sederhana berbasis vocabulary istilah CI umum karena tidak tersedia dataset log sukses dalam jumlah besar untuk melatih model yang lebih kompleks. Selain itu, ditambahkan LLM sebagai lapisan penjelasan untuk menghasilkan ringkasan root cause dan rekomendasi debugging.

Refleksi Kelompok: AI Failure Intelligence for CI/CD Pipeline

3. Jika punya waktu satu bulan penuh dan akses ke production cluster nyata, apa yang akan dilakukan berbeda atau ditambahkan?

Dengan akses ke cluster production dan data log yang lebih banyak, proses filtering dapat ditingkatkan menggunakan vocabulary yang dibangun dari ribuan log sukses sebagaimana diusulkan Li et al. (2025). Knowledge base juga dapat diperluas menggunakan failure log nyata yang telah tervalidasi sehingga kualitas retrieval semakin baik.

Evaluasi dapat dilakukan langsung kepada developer untuk mengukur dampak sistem terhadap waktu diagnosis dan kegunaan laporan yang dihasilkan. Jika jumlah data historis sudah mencukupi, pendekatan prediktif seperti yang diusulkan Saidani et al. (2022) juga dapat mulai diterapkan untuk mendeteksi potensi kegagalan build sebelum pipeline dijalankan.